Pràctica 6: Comunicació entre dispositius IoT mitjançant MQTT

Assignatura de Xarxes

Jan Accensi i Cristian Prado

Desembre 2025

Universitat de Barcelona

Índex

1 Introducció **2**

2 Part 1: Configuració del Broker MQTT (Mosquitto) **2**

3 Part 2: Anàlisi i Configuració de Xarxa **2**
Q1 2
Q2 3
Q3 3

4 Part 3: Desenvolupament del Firmware (Client MQTT) **3**
Q4 3
Q5 4
Q6 4

5 Part 4: Publicació de Dades (Sensor Simulat) **5**
Q7 5

6 Conclusions **7**

A Annex: Codi subscriptor 1 **8**

B Annex: Codi subscriptor 2 (LED) **9**

C Annex: Codi publicador **10**

1 Introducció

Els objectius d'aquesta pràctica es centren en l'aprenentatge i l'aplicació pràctica dels conceptes relacionats amb les comunicacions IoT (*Internet of Things*) mitjançant el protocol MQTT en una xarxa local.

Hem implementat una arquitectura basada en el model Publicació/Subscripció, utilitzant un ordinador com a servidor central (Broker) i plaques de desenvolupament ESP8266 com a nodes clients. Aquesta memòria recull tant la configuració del servidor com la resolució dels problemes tècnics trobats durant el desenvolupament del firmware.

2 Part 1: Configuració del Broker MQTT (Mosquitto)

Per iniciar la xarxa IoT, hem instal · lat i configurat el broker **Mosquitto** en un entorn macOS. La configuració ha estat clau per permetre l'accés extern des de les plaques ESP8266.

Hem editat el fitxer `mosquitto.conf` per establir el *listener* al port estàndard 1883 i hem habilitat l'autenticació mitjançant usuari i contrasenya per afegir una capa de seguretat. S'ha creat l'usuari `cristianprado` i s'ha generat el fitxer de contrasenyes encriptades.

La comanda d'execució utilitzada ha estat:

```
mosquitto -c mosquitto.conf -v
```

L'opció `-v` (*verbose*) ens ha permès monitoritzar en temps real els *logs* de connexió, tal com es veu a la imatge següent, on el broker accepta la connexió de la nostra placa (identificada com a `Huzzah_Subscriptor`).

```
(base) cristianpradomendez@MacBook-Pro-de-Cristian-2 P6 % mosquitto -c mosquitto.conf
1765644087: mosquitto version 2.0.22 starting
1765644087: Config loaded from mosquitto.conf.
1765644087: Opening ipv4 listen socket on port 1883.
1765644087: mosquitto version 2.0.22 running
```

Figura 1: Broker Mosquitto en execució mostrant la negociació de connexió (CONNACK) i publicació de missatges.

3 Part 2: Anàlisi i Configuració de Xarxa

Abans de procedir amb la programació dels dispositius, hem analitzat els conceptes teòrics fonamentals per a la correcta implementació de la xarxa.

Q1. Si volguéssim treballar únicament en connexió local, quina o quines direccions IP podríem utilitzar a la configuració del nostre servidor?

Resposta: Això depèn totalment de qui es vol connectar amb qui. Si estiguéssim fent proves només dins del propi Mac (simulant clients amb terminals), la IP més lògica i ràpida seria la de *loopback* (`127.0.0.1` o `localhost`). En aquest cas, el trànsit no surt mai de la targeta de xarxa.

Però, en el nostre cas real, l'ESP8266 és un dispositiu físic extern que no sap què és el "localhost" del Mac. Per tant, hem hagut d'utilitzar la **IP Privada de la LAN** que el router ha assignat al nostre ordinador (en el nostre cas, la `POSAR LA IP`). Aquesta és l'única adreça "visible" per a la placa dins la xarxa WiFi. Si volguéssim ser més permissius, podríem configurar el servidor amb la IP `0.0.0.0`, que bàsicament li diu al broker que escolti tot el que arribi, vingui d'on vingui.

Q2. WiFiClient admet 2 modes, WIFI_STA (Station) i WIFI_AP (Access point). Quina diferència hi ha entre els 2 modes?

Resposta: La diferència és el rol jeràrquic dins la xarxa. En mode **Station (STA)**, la placa es comporta com un "convidat", exactament igual que el nostre mòbil o portàtil: busca una xarxa existent (la de la universitat o casa), hi posa la contrasenya i demana permís per entrar. Aquest és el mode que hem fet servir perquè necessitem que la placa i el Mac estiguin connectats al mateix router.

En canvi, en mode **Access Point (AP)**, la placa es converteix en l'amfitrió o router. Crea la seva pròpia xarxa WiFi i tu t'hi has de connectar a ella. Això és molt útil per a sensors aïllats al mig del camp o per configurar aparells domòtics la primera vegada, però en aquesta pràctica ens aïllaria del broker, ja que la placa crearia una xarxa paral · lela sense accés al Mac.

Q3. Busca informació sobre la qualitat de servei (QoS) a MQTT i explica com afecten els diferents nivells a l'entrega de missatges.

Resposta: El QoS és el "nivell de compromís" que agafa el protocol per entregar la informació.

- **QoS 0:** El més ràpid però menys fiable ("Fire and forget"). La placa envia la dada i se n'oblida. Si just en aquell moment cau el WiFi, la dada es perd per sempre. Per a un sensor que envia temperatura cada segon, no passa res si en perdem una.
- **QoS 1:** Aquí busquem seguretat. El broker ha de confirmar que ha rebut el missatge (`PUBACK`). Si la placa no rep la confirmació, ho torna a enviar. El "perill" és que si la confirmació es perd pel camí, la placa reenvia la dada i el servidor la rep duplicada.
- **QoS 2:** El nivell màxim ("Exactly once"). Garanteix que arriba una i només una vegada. Fan una negociació de quatre passos molt robusta. És perfecte per a sistemes crítics (pagaments, alarmes), però evidentment és el més lent i consumeix més recursos.

4 Part 3: Desenvolupament del Firmware (Client MQTT)

En aquesta fase hem desenvolupat el codi per a l'ESP8266. Aquest procés no ha estat exempt de problemes, els quals hem hagut de depurar pas a pas.

Q4. Desenvolupa i carrega el firmware a la teva placa que permeti rebre missatges i gestionar la connexió.

Podem trobar el codi desenvolupat a l'Annex A. Comentar que el codi consisteix en utilitzar funcions de connexió a la xarxa que ja havíem utilitzat anteriorment. Cal remarcar que un dels obstacles a l'hora de desenvolupar-ho va ser un error de compilació causat per una errada al guió de la pràctica. El document suggeria utilitzar `wifi_client.mode()`, però el compilador ens indicava que l'objecte client no tenia aquest mètode (Veure Figura **??**a). La solució va ser utilitzar la classe global `WiFi` per a la configuració de la ràdio.

Posteriorment, en carregar el codi, vam observar caràcters il · legibles al monitor sèrie (Figura **??**b). Això indicava una desincronització en la velocitat de transmissió. Ajustant el monitor a **115200 bauds** vam poder llegir correctament els missatges de depuració.

Per tal de testejar que els missatges es reben correctament, utilitzem la comanda:

```
mosquitto_pub -h 192.168.1.114 -t board/led -m "Hello World"
```

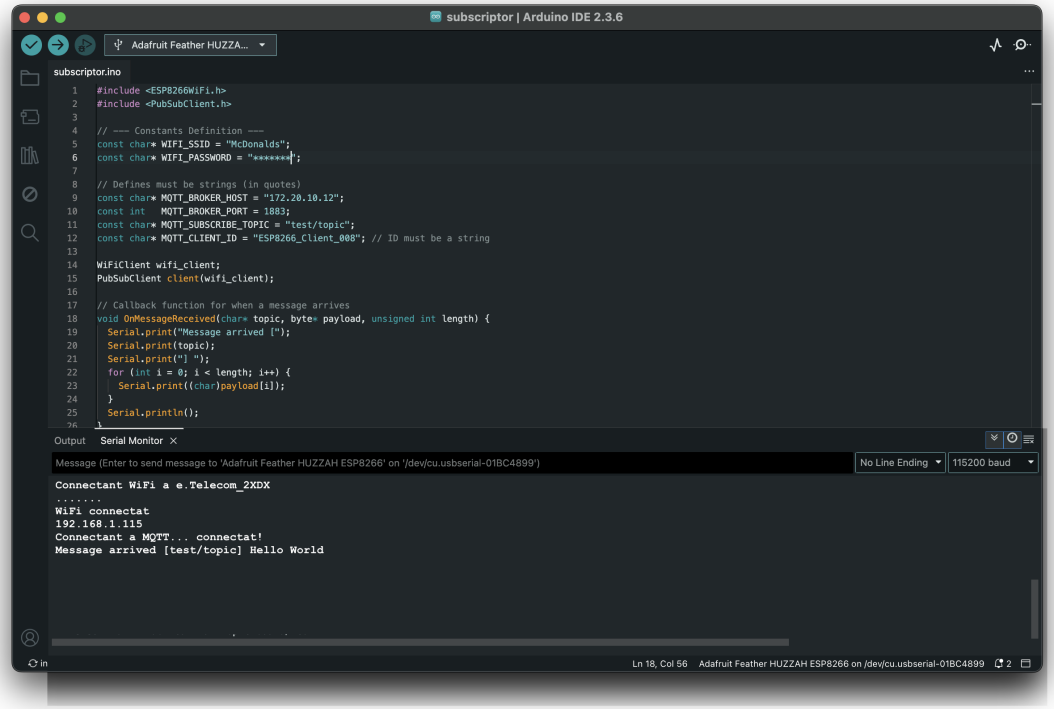

Figura 2: Caption

Q5. La connexió WiFi pot fallar o demorar-se. Imprimeix informació per saber si connectem. (Opcional)

S'ha afegit un bucle `while` al `setup()` que comprova `WiFi.status() != WL_CONNECTED`. Mentre no s'estableix la connexió, s'imprimeixen punts "..."pel port sèrie cada 500ms per indicar a l'usuari que el procés està en curs. A la Figura 3 es pot apreciar aquesta seqüència abans del missatge `Wifi connectat` i la IP assignada `192.168.1.115`.

Q6. Subscriu-te al topic 'board/led', encén el LED quan rebis un nombre inferior a 28 i apaga'l en qualsevol altre cas. (Opcional)

Aquesta funcionalitat requereix transformar el missatge rebut (text) a un valor numèric. Al codi hem implementat la conversió de l'array de bytes a `String` i després a `int`. Com es demostra a la captura següent, quan la placa rep el valor **27**, el firmware detecta que és menor a 28 i activa el LED (imprimint "ENCENENT LED"). Quan rep un **29**, l'apaga. És important mencionar que les proves les hem fet amb el publicador de mosquitto. Concretament hem executat les següents comandes:

```
mosquitto_pub -h 192.168.1.114 -t board/led -m 27
mosquitto_pub -h 192.168.1.114 -t board/led -m 29
```

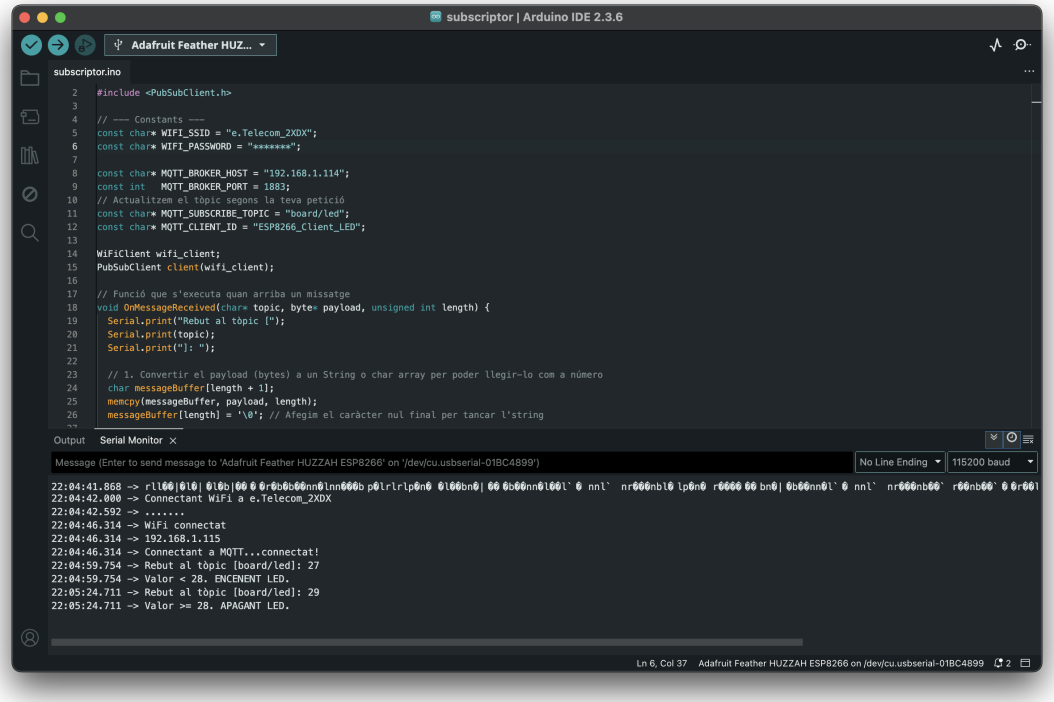

Figura 3: Monitor Sèrie mostrant la recepció de valors i la decisió lògica de control del LED.

5 Part 4: Publicació de Dades (Sensor Simulat)

Finalment, hem configurat la placa perquè actuï com a emissor de dades (Publicador), tancant així el cicle de comunicació IoT.

Q7. Desenvolupa i carrega el firmware a la teva placa que enviï cada 5 segons un nombre a un topic.

Hem programat la placa perquè generi un nombre aleatori (simulant un sensor) i l'enviï periòdicament. Per tal de generar el valor utilitzem la funció random entre 2 constants MIN i MAX.

S'ha verificat el funcionament bidireccional: la placa publica dades de sensors (Part 3) mentre simultàniament escolta ordres per al LED (Part 2), gràcies a l'arquitectura no bloquejant del codi. Els missatges de depuració al Monitor Sèrie confirmen l'enviament i la recepció correcte de paquets.

Per verificar que les dades realment viatgen per la xarxa i no es queden a la placa, hem fet servir el terminal del Mac com a "receptor testimoni". Amb la comanda:

```
mosquitto_sub -h "192.168.1.114" -t board/led
```

A les imatges inferiors es veu la prova definitiva de funcionament:

1. **L'Arduino (Emissor):** Publica els valors 33, 24, 21... al tòpic `board/led`.
2. **El Mac (Receptor):** Rep exactament els mateixos valors via WiFi a través del broker.

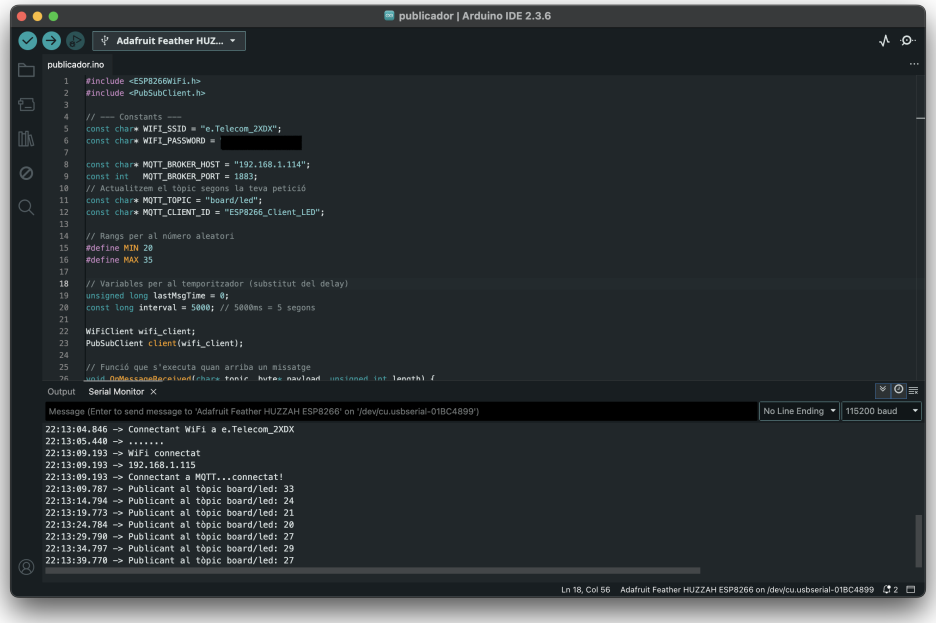

Figura 4: Arduino publicant dades cada 5s.

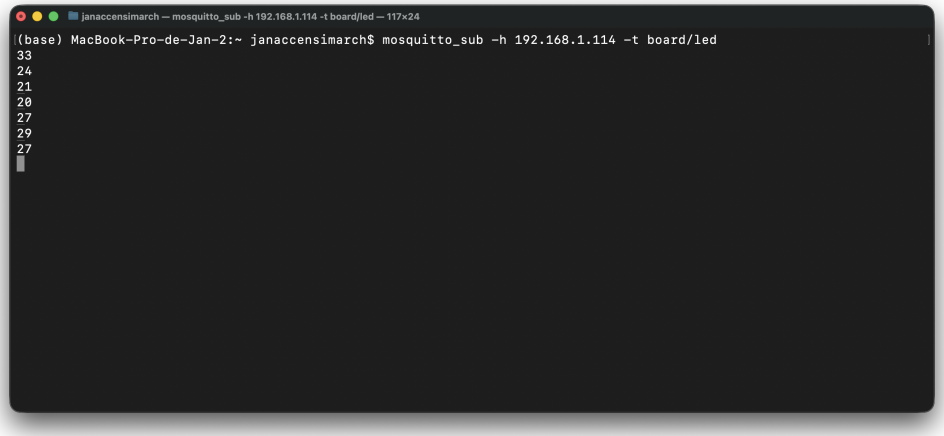

Figura 5: Terminal del Mac rebent les dades.

6 Conclusions

La realització d'aquesta pràctica ens ha permès consolidar els conceptes fonamentals de l'Internet de les Coses (IoT), passant de la teoria a una implementació real i funcional. Hem comprovat de primera mà la eficiència del model **Publicació/Subscripció** respecte als models tradicionals client-servidor. Hem vist com aquest paradigma permet desacoblar els dispositius: el sensor no necessita saber qui rebrà la dada, i l'actuador no necessita saber qui l'ha enviat, sent el Broker l'encarregat de gestionar aquest trànsit de manera transparent i ràpida.

Un dels aprenentatges clau ha estat la capacitat de transformar dades digitals en accions físiques. La implementació del control del LED mitjançant missatges MQTT ens ha demostrat la baixa latència del protocol: la resposta de la placa ha estat pràcticament instantània des del moment de la publicació al terminal del Mac. Això valida l'ús d'MQTT per a aplicacions de domòtica i control en temps real on la velocitat de reacció és crítica.

Finalment, haver integrat tant la funció de sensor (publicador) com la d'actuador (subscriptor) en un mateix dispositiu ens ha ofert una visió completa del cicle de vida de la dada. Hem après que, en un entorn IoT, la complexitat no resideix només en la programació del microcontrolador, sinó en la correcta orquestració de la xarxa, la gestió de permisos al Broker i l'estructura dels tòpics per mantenir un sistema ordenat i escalable.

A Annex: Codi subscriptor 1

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// --- Constants Definition ---
const char* WIFI_SSID = "McDonalds";
const char* WIFI_PASSWORD = "********";

// Defines must be strings (in quotes)
const char* MQTT_BROKER_HOST = "172.20.10.12";
const int   MQTT_BROKER_PORT = 1883;
const char* MQTT_SUBSCRIBE_TOPIC = "test/topic";
const char* MQTT_CLIENT_ID = "ESP8266_Client_008"; // ID must be a string

WiFiClient wifi_client;
PubSubClient client(wifi_client);

// Callback function for when a message arrives
void OnMessageReceived(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}

// Function to connect and reconnect to MQTT
void reconnect() {
  // Loop until we're reconnected
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");

    // Attempt to connect using the Client ID
    if (client.connect(MQTT_CLIENT_ID)) {
      Serial.println("connected");
      // Once connected, resubscribe
      client.subscribe(MQTT_SUBSCRIBE_TOPIC);
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  delay(10);

  // Connect to WiFi
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

```

```
  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());

  // Setup MQTT
  client.setServer(MQTT_BROKER_HOST, MQTT_BROKER_PORT);
  client.setCallback(OnMessageReceived);
}

void loop() {
  // If MQTT is not connected, run the reconnect function
  if (!client.connected()) {
    reconnect();
  }
  // This must be called regularly to process incoming messages and maintain the connection
  client.loop();
}
```

Listing 1: Firmware ESP8266 Complet

B Annex: Codi subscriptor 2 (LED)

```

#include <ESP8266WiFi.h>
#include <PubSubClient.h>

// --- Constants ---
const char* WIFI_SSID = "e.Telecom_2XDX";
const char* WIFI_PASSWORD = "*******";

const char* MQTT_BROKER_HOST = "192.168.1.114";
const int   MQTT_BROKER_PORT = 1883;
// Actualitzem el t pic segons la teva petici
const char* MQTT_SUBSCRIBE_TOPIC = "board/led";
const char* MQTT_CLIENT_ID = "ESP8266_Client_LED";

WiFiClient wifi_client;
PubSubClient client(wifi_client);

// Funci  que s'executa quan arriba un missatge
void OnMessageReceived(char* topic, byte* payload, unsigned int length) {
  Serial.print("Rebut al t pic [");
  Serial.print(topic);
  Serial.print("]: ");

  // 1. Convertir el payload (bytes) a un String o char array per poder llegir-lo com a
    n mero
  char messageBuffer[length + 1];
  memcpy(messageBuffer, payload, length);
  messageBuffer[length] = '\0'; // Afegim el car cter nul final per tancar l'string

  String message = String(messageBuffer);
  Serial.println(message);

  // 2. Convertir l'String a un n mero enter (int)
  int valorRebut = message.toInt();

  // 3. Aplicar la l gica: Encendre si  s  < 28
  if (valorRebut < 28) {
    Serial.println("Valor < 28. ENCENENT LED.");
    digitalWrite(BUILTIN_LED, LOW); // LOW enc n el LED a l'ESP8266
  } else {
    Serial.println("Valor >= 28. APAGANT LED.");
    digitalWrite(BUILTIN_LED, HIGH); // HIGH apaga el LED a l'ESP8266
```

```
  }
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connectant a MQTT...");
    if (client.connect(MQTT_CLIENT_ID)) {
      Serial.println("connectat!");
      client.subscribe(MQTT_SUBSCRIBE_TOPIC); // Ens subscrivim a 'board/led'
    } else {
      Serial.print("error, rc=");
      Serial.print(client.state());
      Serial.println(" reintentant en 5 segons");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  // --- CONFIGURACI  LED ---
  pinMode(BUILTIN_LED, OUTPUT);     // Inicialitzem el pin del LED
  digitalWrite(BUILTIN_LED, HIGH);  // Assegurem que comenci apagat (HIGH = OFF)
  // ------------------------

  delay(10);
  Serial.println();
  Serial.print("Connectant WiFi a ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connectat");
  Serial.println(WiFi.localIP());

  client.setServer(MQTT_BROKER_HOST, MQTT_BROKER_PORT);
  client.setCallback(OnMessageReceived);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
}
```

C Annex: Codi publicador

```
    #include <ESP8266WiFi.h>
#include <PubSubClient.h>

// --- Constants ---
const char* WIFI_SSID = "e.Telecom_2XDX";
const char* WIFI_PASSWORD = "*******";

const char* MQTT_BROKER_HOST = "192.168.1.114";
const int   MQTT_BROKER_PORT = 1883;
// Actualitzem el t pic segons la teva petici
```

```
const char* MQTT_TOPIC = "board/led";
const char* MQTT_CLIENT_ID = "ESP8266_Client_LED";

// Rangs per al n mero aleatori
#define MIN 20
#define MAX 35

// Variables per al temporitzador (substitut del delay)
unsigned long lastMsgTime = 0;
const long interval = 5000; // 5000ms = 5 segons

WiFiClient wifi_client;
PubSubClient client(wifi_client);

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connectant a MQTT...");
    if (client.connect(MQTT_CLIENT_ID)) {
      Serial.println("connectat!");
      client.subscribe(MQTT_TOPIC); // Ens subscrivim a 'board/led'
    } else {
      Serial.print("error, rc=");
      Serial.print(client.state());
      Serial.println(" reintentant en 5 segons");
      delay(5000);
    }
  }
}


void setup() {
  Serial.begin(115200);

  delay(10);
  randomSeed(micros()); // Inicialitzar la llavor aleat ria perqu  els n meros vari n

  Serial.println();
  Serial.print("Connectant WiFi a ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connectat");
  Serial.println(WiFi.localIP());

  client.setServer(MQTT_BROKER_HOST, MQTT_BROKER_PORT);
}

void loop() {
  // 1. Assegurar connexi
  if (!client.connected()) {
    reconnect();
  }
  client.loop(); // IMPORTANT: Aix  mant  l'MQTT viu

  // 2. Temporitzador no bloquejant (cada 5 segons)
  unsigned long now = millis();
  if (now - lastMsgTime > interval) {
    lastMsgTime = now; // Actualitzem el temps

    // Generar dada simulada
    int r = random(MIN, MAX);
```

```

    // Convertir int a String i despr s a char array (c_str)
    // Aquesta  s  la forma correcta de fer-ho:
    String payload = String(r);

    Serial.print("Publicant al t pic ");
    Serial.print(MQTT_TOPIC);
    Serial.print(": ");
    Serial.println(payload);

    // Enviar missatge
    client.publish(MQTT_TOPIC, payload.c_str());
  }

}
```